How an AI Economy Behaves: A Multi-Agent Simulation in Which AI Companies Learn to Compete by Reinforcement Learning

Ravi Kadam
Independent Researcher
ravindrakad@gmail.com

October 2026

Abstract

We present an open, reproducible simulation of a stylised AI industry over ten years and let AI product companies learn to compete in it with reinforcement learning (RL). The simulated economy links a chip fabricator, chip designers, data centres, frontier and open-weight model providers, 5–12 competing AI product companies and 300 enterprise customers in three sectors. Companies decide which use cases to pursue, which model to build on, how much to spend, and when to pilot, launch, scale or quit, facing uncertain technology, hidden customer value, compute shortages and rivals. Learning from scratch failed: the agent stopped exploring and lost most of its capital. Imitating a competent rule and then improving with Proximal Policy Optimisation succeeded in under eight minutes on a laptop. Across five independently trained agents evaluated on 100 unseen markets, the agents created $15.5M of value per company on average against $14.0M for the best rule-based strategy; three of five were significantly better and none worse, mainly by choosing harder, less crowded use cases and timing pilots to customer readiness. Under a severe compute crunch, all five trailed the rule and survived less often. An apparently decisive single-run advantage (+$5.9M) shrank to +$1.5M once training and action-sampling randomness were accounted for. In the simulated market, adoption follows S-curves, a regulated sector is held back by reliability rather than demand, first launchers gain no lasting share, and supply shocks quadruple compute prices. Written for readers new to RL, the paper explains each RL concept alongside the result it produces and states nine hypotheses about real AI markets, each tied to the modelled assumptions it depends on and the evidence needed to test it. Code, trained agents and a live viewer are open source.

1 Introduction

The AI industry is a chain of businesses that depend on one another, and the chain behaves in ways no single company controls. Chip factories decide how much capacity to build years ahead. Data centres buy those chips and rent out computing power. Model labs sell access to AI models. Product companies build AI applications on those models, and enterprises decide whether the applications are worth paying for. A shortage, a breakthrough or a wave of failed projects at any point ripples through all the others.

These ripples are hard to reason about and impossible to experiment with in reality. Nobody can rerun the last five years with one chip factory fewer, or with model progress twice as fast. A simulation can: it lets us build a small, transparent version of the industry, change one assumption at a time, and watch what follows.

This paper describes such a simulation, the *AI Economy Lab*, and uses it for two purposes:

1. **To understand market dynamics.** How does enterprise adoption spread? What does a supply shock do to prices, failures and competition? Does being first to market pay?
2. **To study decision-making under uncertainty.** We let software agents learn, by trial and error, how to run an AI product company in this market, using reinforcement learning. We then ask what strategies they discover and whether they beat sensible hand-written rules.

No background in reinforcement learning or economics is assumed. Sections 3 and 4 describe the scenario and its rules. Section 5 introduces the competing companies. Section 6 presents the results and explains each RL idea where it matters. Section 7 separates what the simulation shows from what it might imply for real markets, stating the latter as testable hypotheses.

Our contributions are: (i) an open simulator of a multi-layer AI value chain with audited accounting and fourteen acceptance tests; (ii) a masked, autoregressive actor-critic trained by behaviour cloning followed by PPO on consumer hardware, with a documented failure of pure RL; (iii) a paired evaluation of five independently trained agents on 100 unseen markets that separates market, training and action-sampling randomness, plus shifted regimes and six ablations; and (iv) an educational treatment that connects each RL concept to an observable result. Every parameter is an illustrative assumption chosen to be plausible, not a measurement of the real industry; the value of the exercise lies in the mechanisms it makes visible.

2 Related work

Agent-based economics. Agent-based computational economics studies markets as systems of interacting, boundedly rational agents rather than as equilibria [6, 22]. Our simulator follows this tradition with a stylised multi-layer value chain, adding learning agents to one layer.

Reinforcement learning in economic settings. The AI Economist used two-level multi-agent RL to design tax policy in a simulated economy [27], and RL pricing agents have been shown to learn collusive behaviour [3]. We focus instead on firm strategy in a technology market with stage-gated projects, uncertain feasibility and supply-chain delays.

Methods. We use Proximal Policy Optimisation [17] with generalised advantage estimation [16]; PPO with parameter sharing is a strong baseline in multi-agent settings [26]. Initialising RL from imitation of demonstrations goes back to ALVINN [13] and was central to AlphaGo [18]; imitation's compounding errors are analysed by Ross et al. [15]. Training against frozen past versions reduces non-stationarity in multi-agent learning [25]. Our reward correction follows potential-based shaping [11], and we mask invalid actions [8]. Standard RL texts cover the foundations [20].

Generalisation and evaluation. RL agents overfit to training environments [4]; training on randomised conditions is a common remedy [23]. Results in deep RL are noisy and need many independent evaluations with confidence intervals [1, 7], a point our own results illustrate.

Market dynamics. Our supply chain exhibits delayed, amplified responses familiar from the bullwhip effect [9] and from experiments on misperceived feedback [19]. Adoption follows the S-shaped diffusion described by Bass [2] and Rogers [14]. We test first-mover advantage [10] in a market where all firms share the same underlying models. Business simulation games have long been used for management education [5]; our live viewer serves a similar purpose.

3 The scenario

The simulation follows a stylised AI industry for ten years, one month at a time. Six kinds of players form a supply chain (Figure 1). Only one kind, the AI product companies, makes the

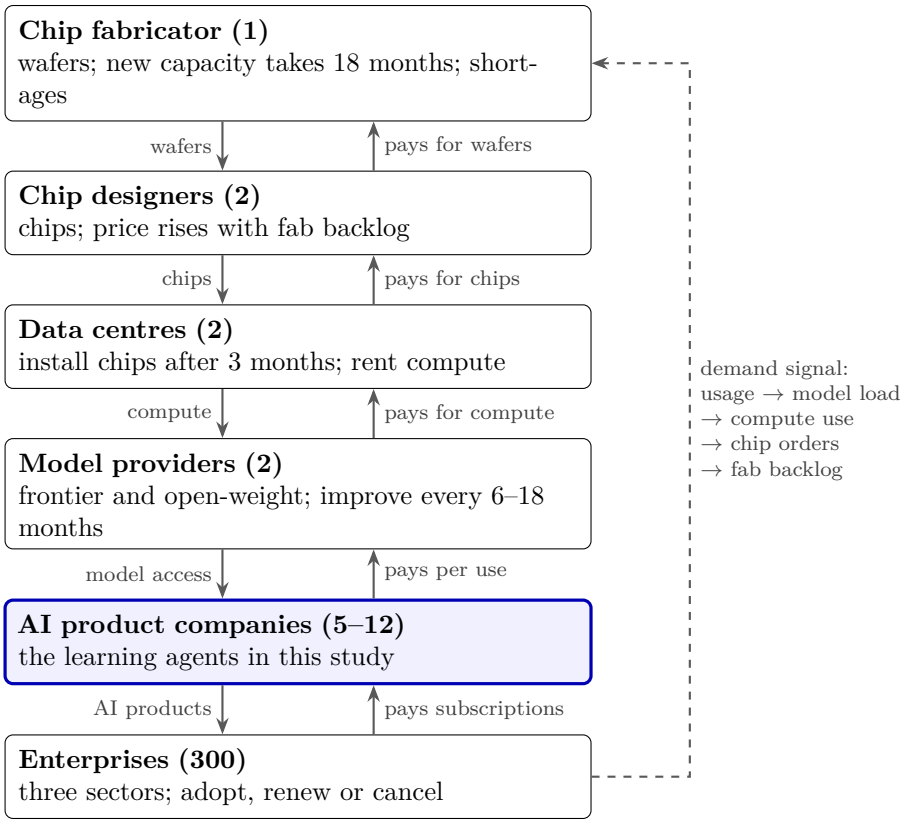


Figure 1: The simulated AI industry. Goods flow down the chain; money and the demand signal flow back up, each with its own delays.

strategic choices this study is about; the others follow fixed rules designed to behave like their real counterparts.

Upstream suppliers. One fabricator produces wafers; two chip designers turn them into chips of different efficiency and price; two data centres buy the chips and sell computing power in GPU-equivalent units. A data centre orders chips after it has been more than 80% busy; chips arrive months later; a new fab line takes 18 months. From time to time the fab loses 30–60% of its output for 3–9 months.

Model providers. A *frontier* model is more capable and more reliable but expensive; an *open-weight* model is cheaper but weaker. Both improve at random intervals of 6–18 months, occasionally in a large jump, and the open model closes part of the gap at each release. Capability is tracked per customer sector.

AI product companies. Each starts with $1.5M in cash and a $400k credit line, can run at most three projects, and has a skill level and a risk appetite (cautious, balanced or aggressive). New companies enter from month 12 when incumbents are profitable, up to 12 in the market; companies that cannot pay their bills go bankrupt.

Enterprises. Three hundred enterprises in three sectors that value different things (Table 1). Each enterprise has a *confidence* in AI products (0–1) that rises with successful pilots, falls with failures and cancellations, and spreads within its sector.

Use cases. The market contains 60 use cases: specific jobs AI could do for one sector. Twenty-four are visible at the start and the rest are discovered over time, faster once models can tackle them. Each has a hidden difficulty, a value per customer that companies see only as a noisy estimate, a

Sector	Firms	AI budget	Values most	Reliability needed	Confidence
Customer operations	110	\$40k/month	price	70%	0.55
Software engineering	100	\$60k/month	capability	70%	0.60
Regulated industries	90	\$80k/month	reliability	84% + per use case	0.40

Table 1: Enterprise sectors.

number of enterprises that need it (8–35% of the sector), a development time of 2–5 months per stage, an integration burden, an optional extra reliability requirement and a compute footprint per customer.

An illustrative journey. Suppose a software-engineering use case is worth \$18k a month to each of 30 enterprises. A company sees a difficulty estimate of 0.62 (true value 0.55, hidden). It starts a project on the frontier model with a standard budget (\$20k, then \$25k a month). After about three months of research and three of prototyping it faces the technical gate, with a success chance of about 82%. It then pilots with three enterprises for two months and passes the commercial gate. It launches at about \$5.8k a month per customer; model usage costs about \$1.6k and support \$0.8k, leaving \$3.4k of margin. Limited to about three new customers a month, it reaches 20 customers and about \$68k monthly profit after seven months, until a rival competes or compute prices rise.

4 The rules of the simulation

4.1 The monthly cycle

Every month runs the same steps in the same order, so the same seed and decisions reproduce exactly the same history: (1) random shocks and queued arrivals; (2) publication of prices, capacity and confidence with a one-month lag; (3) simultaneous decisions by all companies, which cannot see rivals' current choices; (4) budget checks and fair random allocation of contested pilot customers; (5) project progress and gate draws; (6) adoption, cancellation and compute service; (7) settlement of costs, interest and bankruptcy; (8) price, confidence, supplier and entry updates; (9) scoring.

4.2 Two kinds of uncertainty

The **technical gate** asks whether the AI can do the job. Its success probability is

$$p = \mathrm{clip}_{[0.03,\,0.97]}\ \sigma\big(6\,(c_{m,s} - d) + \mathrm{skill} + \mathrm{effort} - 1.5\,b + 0.4\big), \tag{1}$$

where $c_{m,s}$ is the chosen model's capability in the sector, d the hidden difficulty, b the integration burden and σ the logistic function. For a typical company, a model 0.2 weaker than the task gives about 50%, a matched model about 77%, and a model 0.2 stronger about 92%. Failed projects can be retried at extra cost, and each attempt sharpens the company's difficulty estimate.

The **commercial gate** follows a two-month paid pilot with up to three enterprises. It passes only if pilot customers received more value than they paid plus their required return, and the model met their reliability requirement. When a sector's confidence is low, enterprises may refuse to pilot at all; this *deferral* is recorded separately from failure.

4.3 Buying decisions, compute and supply

Each month every enterprise with an unmet need and budget compares offers, weighing expected benefit (rising with capability and reliability) against price and integration cost, scaled by its confidence. It picks the best offer, and each extra competitor lowers list prices by 12%. Contracts take 1–4 months to sign, and a company onboards about three customers a month (nine after a $150k scale-up). Customers who receive less value than expected are much more likely to cancel, and visible cancellations lower sector confidence.

Every customer consumes compute. When demand exceeds installed capacity, all buyers receive the same reduced share; unserved usage is not billed and raises cancellations. The compute price moves with utilisation between $3 and $30 per unit. Data centres order chips above 80% utilisation, with about four months to install; the fab invests when its backlog exceeds 1.2 months of output, with an 18-month lead time.

4.4 Money, value and information

All money moves through a double-entry ledger whose balances always sum to zero. Main company costs are overhead ($25k/month), development per project ($12k, $25k or $50k per month for lean, standard or premium budgets), starting ($20k), piloting ($40k), launching ($30k), scaling ($150k) and support ($0.8k per customer per month), plus model usage. Credit costs 1.2% a month.

A company's **value** is its cash minus debt, plus the present value of its live products' recent profit, plus a cautious book value for projects in development. Its monthly change is the company's score and, in Section 6, the RL reward.

Companies observe their own state, noisy estimates of use-case difficulty and value, lagged model capability, reliability and prices, sector confidence, and rivals' launches. True difficulty, enterprise-specific benefit, rivals' private decisions, future shocks and gate draws are hidden. Fourteen automated tests confirm determinism, the absence of revenue without demand, persistence of technical failure under unlimited compute, no adoption without innovation, price rises under scarcity, respect for capacity lead times, a balanced ledger and the absence of information leakage.

5 The competitors

Each month every company takes one action: hold, start a project (choosing use case, model and budget), pilot, launch, scale, retry, pause or abandon. Five kinds of decision-maker compete (Table 2). All rule-based strategies share sensible habits: they pilot and launch ready projects, retry or abandon failures, close persistent loss-makers and scale profitable products.

Strategy	How it decides
Random	Any allowed action at random (a control).
Easy-first	Starts the use case that looks easiest, on the frontier model with a standard budget.
Best expected value	Scores every use case, model and budget by a hand-built estimate of success odds × profit and picks the best.
Cash-preserving	One project at a time, open model, lean budget, only with 12 months of cash in hand.
Learning agent	A neural network that improves by playing thousands of simulated markets (reinforcement learning).

Table 2: The competing decision-makers.

The learning agent sees what a real company could: its finances and projects, a shortlist of 12 use cases with noisy estimates, and last month's market data (415 numbers in total). One network controls every learning company in a market, so it also learns from playing against copies of itself.

6 Results, and the reinforcement learning behind them

6.1 Trial and error alone fails

Our first attempt to train an agent from scratch learned to do almost nothing and still lost most of its money. Understanding why is a good introduction to reinforcement learning.

The basic loop. RL trains an *agent* (here, the decision-maker for one company) by letting it act in an *environment* (the simulated market). Each month the agent receives an *observation*, chooses an *action*, and receives a *reward*: the change in its company's value. Its goal is to maximise total reward over the ten-year episode, not just the next month [20].

The policy. The agent's decision rule is its *policy*: a neural network mapping observations to a probability for each action. Training nudges the network so that actions followed by higher-than-expected reward become more likely. Repeated over millions of decisions, good behaviour can emerge without anyone writing it down.

Exploration. An agent that only repeats what has worked never discovers anything better, so early on it must try things. The randomness of its choices is measured by *entropy*; training gradually lowers it as the agent gains confidence.

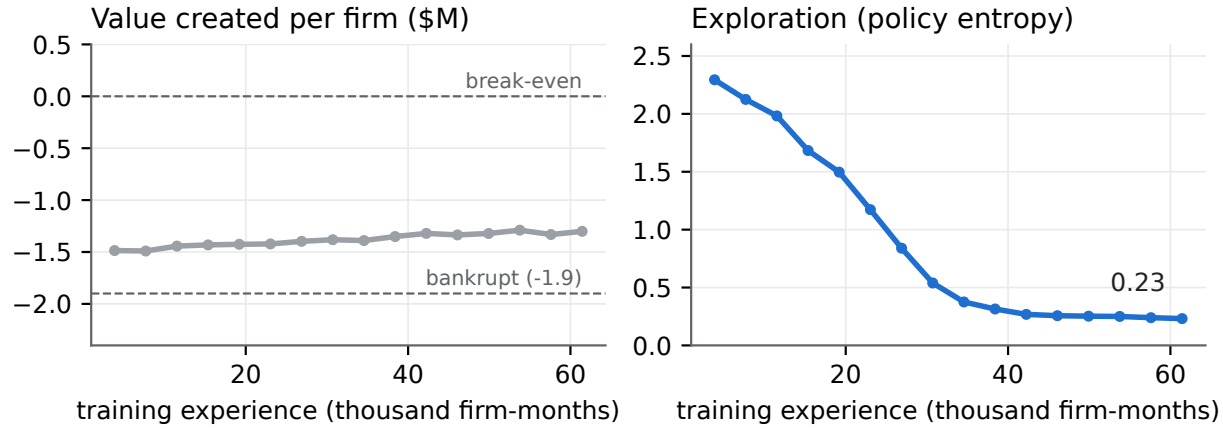


Figure 2: Learning from scratch. Value created stays near total loss while exploration (entropy) collapses from 2.3 to 0.2 within 61,000 simulated firm-months.

Why it failed. Every investment looks like a loss for months. A project costs money immediately; the payoff arrives only after a correct chain of decisions (start, wait, pilot, wait, launch) 8–12 months later, and only if both gates pass. An agent acting randomly almost never completes that chain, so it mostly sees spending followed by lower value. This is the *credit-assignment problem*: when a reward finally arrives, which earlier action deserves the credit? The agent concluded that spending is bad, stopped exploring, and held still while overhead drained its cash (Figure 2). Organisations face the same trap when they judge a new kind of project by its first quarter's cash flow.

6.2 Learn from an expert, then improve

Imitation. We recorded 77,398 decisions of the best-expected-value rule across 64 markets and trained the network to predict them from the same observations, a technique called *behaviour cloning* [13]. After six passes it matched the expert's choice of action type 97.8% of the time, giving the agent a competent starting point instead of random behaviour.

A fairer score. We count money spent on in-development projects as a partly recoverable asset, written off only on failure or abandonment. The episode total is unchanged but investment no longer looks like an immediate loss. This is a form of potential-based *reward shaping* [11], which changes when credit arrives without changing what counts as winning.

PPO. The agent then played 32 independent markets at once across 8 CPU cores, in 60-month rollouts, and updated its policy with Proximal Policy Optimisation [17]. Alongside the policy, the network learns a *value function*: a prediction of future reward from the current situation. Each action is judged by its *advantage*, how much better things turned out than predicted [16], which separates good decisions from good luck. PPO also limits how far the policy may change per update, keeping improvement steady. Rewards are discounted by 0.99 per month, so a reward about six years away counts half as much as one today.



Figure 3: After imitation, PPO raises value created on five held-out development markets from 13.9 to a best of 22.4 $M per firm; training stopped after eight checks without improvement.

Held-out testing. Agents can overfit to their training markets [4]. Every five updates we tested on five markets never used for training, kept the best version and stopped after eight checks without improvement (Figure 3). Final evaluation uses still other markets. *Action masks* remove impossible choices before sampling [8], and one network with shared parameters controls every learning company, with each company's risk appetite as an input [26]. Seats in training markets were 60% learning agent, 30% rule-based and 10% frozen past versions of the agent [25]. The original training run took 7.7 minutes on an Apple M2 Pro (four repeat runs took 2.7–7.0 minutes; Section 6.4): a 16,384-sample update ran 1.5× faster on the GPU (0.58 s against 0.88 s), while small per-step inference batches ran faster on the CPU.

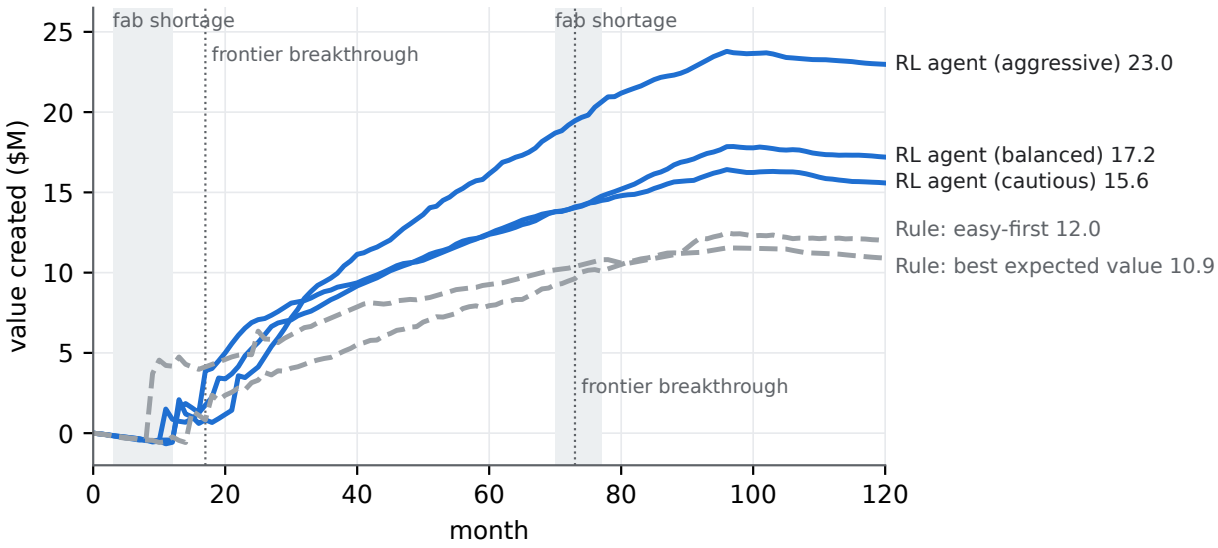


Figure 4: One 120-month market (seed 47) with three agent-run and two rule-based founding companies. Shaded: fab shortages; dotted: frontier-model breakthroughs.

6.3 One game, month by month

In one market (Figure 4), the best-expected-value company launched first, in month 9, and briefly led; the agent's companies launched on less crowded use cases four months later and passed it by month 24. The early shortage barely mattered because little was being sold; the later one slowed everyone. Three later entrants went bankrupt. The aggressive agent company earned the most, although the agent was never told what "aggressive" means; it learned how a bolder company should behave from the risk-appetite input. Values flatten in the final years because only profit that can still be earned before the episode ends is counted. One game proves little: across 20 such games the agent's companies out-earned the rule-based ones in 12, by $12.0M to $8.5M on average.

6.4 Does the learner beat the rules, and does training reliably produce one that does?

A fair test. In each test market one company's seat is given in turn to each strategy, while the other four companies and all entrants follow the same fixed rule-based mix. Because a market's randomness depends only on its seed, every strategy faces identical hidden difficulties, customer needs and shocks: a paired comparison. No test market was used in training.

Three sources of randomness. Results vary across markets, across training runs, and across the agent's own action sampling, since the agent draws each action from its policy's probabilities. Testing one trained agent on many markets measures only the first. We therefore trained five agents with the same procedure and different seeds; each stopped early under the same held-out rule after 86,000–278,000 firm-months (2.7–7.7 minutes). Each agent was evaluated on the same 100 unseen markets (seeds 300–399), with each market's result averaged over three independent draws of the agent's action randomness. The rule-based strategies are deterministic given the market.

In the base market, the agents created $15.5M per company on average against $14.0M for the best rule (Table 3). The average lead is +1.5 $M with a standard deviation of 1.5 $M across runs (range −0.1 to +2.9). Three of the five agents are significantly better than the rule, two are

Strategy (base market, 100 unseen markets)	Value created per company	Survival at month 120
Learning agent, mean of 5 training runs	\$15.5M (range 14.0–17.0)	97%
Best expected value	\$14.0M	99%
Cash-preserving	\$7.2M	77%
Easy-first	\$3.1M	64%
Random	−\$1.8M	0%

Table 3: Focal-seat results on seeds 300–399. A company starts with \$1.5M; −\$1.9M means it lost everything including its credit line.

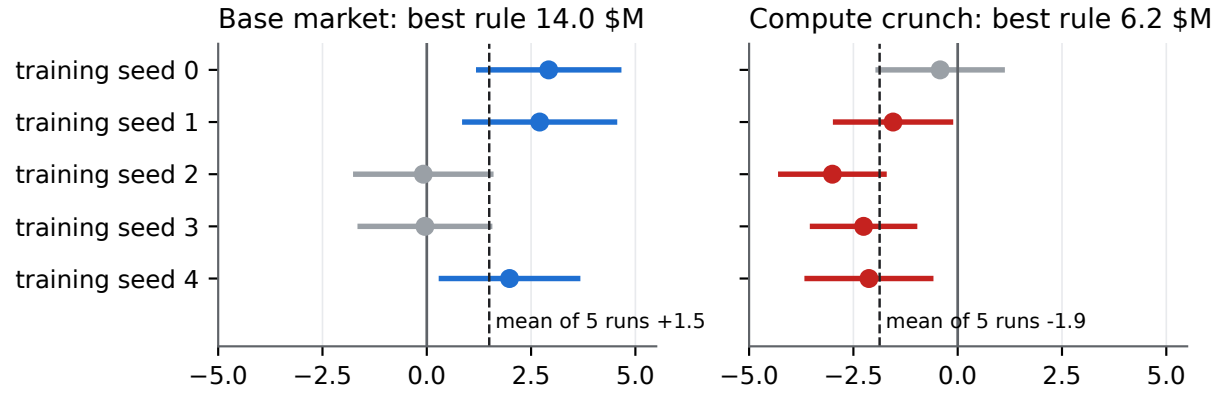


Figure 5: Each of five independently trained agents against the best-expected-value rule on the same 100 unseen markets. Blue: significantly better than the rule; red: significantly worse; grey: indistinguishable. Dashed line: mean across runs.

indistinguishable from it, and none is worse (Figure 5). Training therefore reliably produces an agent at least as good as the strongest hand-written rule, usually somewhat better, but not reliably much better. In the compute crunch every agent trails the rule (average −1.9 \$M, range −3.0 to −0.4), four of the five significantly, and they survive less often (50–62% against 79%).

A lesson about evaluation. An earlier version of this analysis evaluated only the original agent (training seed 0) with a single draw of its action randomness and reported $+5.9 \pm 2.5$ \$M, an apparently decisive result. Averaged over three draws, the same agent's lead is $+2.9 \pm 1.7$ \$M, and action sampling alone changes a single market's outcome with a standard deviation of \$6.6M. With only 30 markets, the first estimate was $+4.1 \pm 4.8$ \$M. Each additional source of randomness we accounted for shrank the apparent advantage, as the RL evaluation literature warns [1, 7].

Other regimes (exploratory). The original agent, evaluated with a single draw on 30 markets per regime, led the best rule by +0.8 to +4.9 \$M under faster or slower technology progress, harder use cases and a wave of entrants, with every interval including zero. These regimes were not repeated across training seeds, so we treat them as exploratory.

The analyses in Sections 6.5 and 6.6 use the original agent (training seed 0), one of the stronger runs. They describe how that agent behaves, not every run.

Behaviour (base market, mixed population)	Agent	Best exp. value	Cash-pres.	Easy-first
Value created per company	$9.4M	$4.0M	$7.8M	$1.5M
Survival at month 120	93%	68%	83%	63%
True difficulty of use cases started	0.61	0.52	0.49	0.39
Starts in the regulated sector	31%	14%	3%	30%
Starts on the frontier model	53%	46%	0%	100%
Budget lean / standard / premium (%)	11/63/26	38/19/44	100/0/0	0/100/0
Pilots refused (customers not ready) per company	1.6	7.2	4.2	0.0
Technical failures per company	1.6	0.7	1.2	0.6
Decisions spent scaling a product	0%	1.7%	1.6%	2.7%

Table 4: Behaviour of the original agent (training seed 0) in 30 base markets with two agent companies, one of each rule, and entrants split between the agent and best expected value (448 companies).

6.5 The strategy the learner discovered

Table 4 shows three patterns. First, the agent fails more and earns more: it suffered more than twice the technical failures of the best-expected-value rule but aimed at harder, less crowded, more valuable use cases, including regulated ones the rule avoided. The rule scores use cases in isolation; the agent learned that a contested use case is worth less than its spreadsheet value. Second, it learned timing: the rule offered pilots to sectors not yet ready and was refused 7.2 times per company, while the agent waited. Third, it never paid to scale up, either a learning gap or a sign that scaling is overpriced here.

6.6 Which forces matter

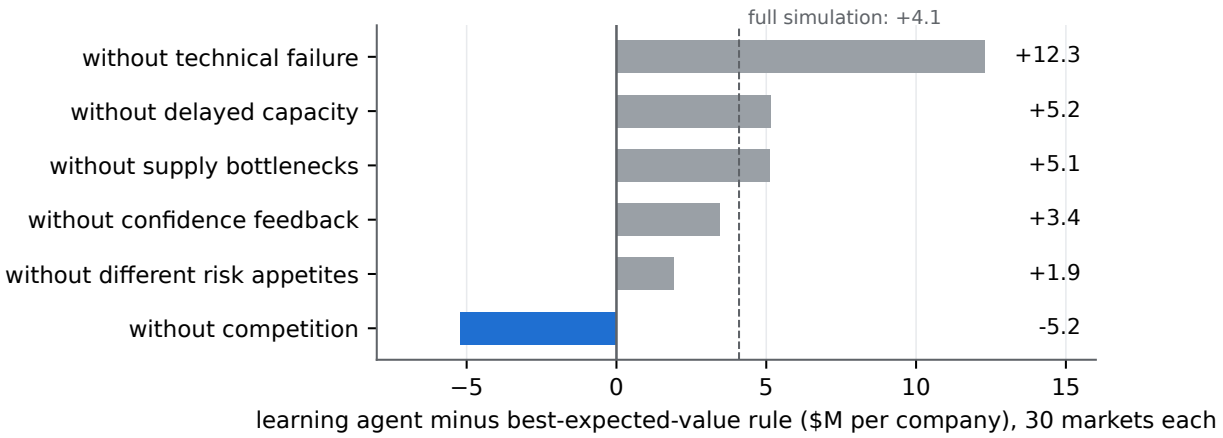


Figure 6: Ablations: original agent (training seed 0) minus best-expected-value rule with one mechanism switched off (30 markets each, one sampling draw). The dashed line marks the same comparison in the full simulation.

An *ablation* removes one part of a system and measures what changes. With the trained agent unchanged, we switched off one mechanism at a time (Figure 6). Without competition (each use case claimable by one company), the best-expected-value rule earned $23.5M against the agent's

$18.3M: the agent's habit of avoiding crowded use cases only pays when rivals exist. Without technical failure, the agent's lead grew to $12.3M, as its appetite for hard use cases paid off in full. Removing supply bottlenecks or delayed capacity widened the lead by about $1M; removing confidence feedback or risk heterogeneity narrowed it slightly. Those four intervals span about ±$5M, so only the competition and technical-failure effects are large relative to noise. Learned strategies are tuned to the world they were trained in.

6.7 How the market behaves

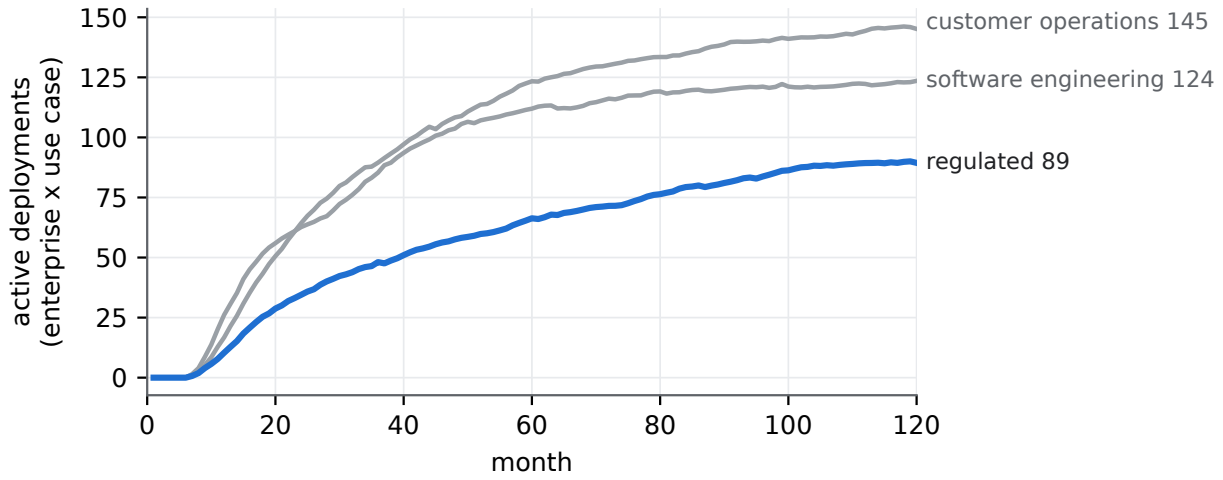


Figure 7: Active deployments by sector, mean of 30 base markets. One deployment is one enterprise subscribed to one use case, so a sector's count can exceed its roughly 20 use cases. Possible deployments per sector (enterprises with a need, summed over that sector's use cases) average 481, 413 and 384 for customer operations, software engineering and regulated industries.

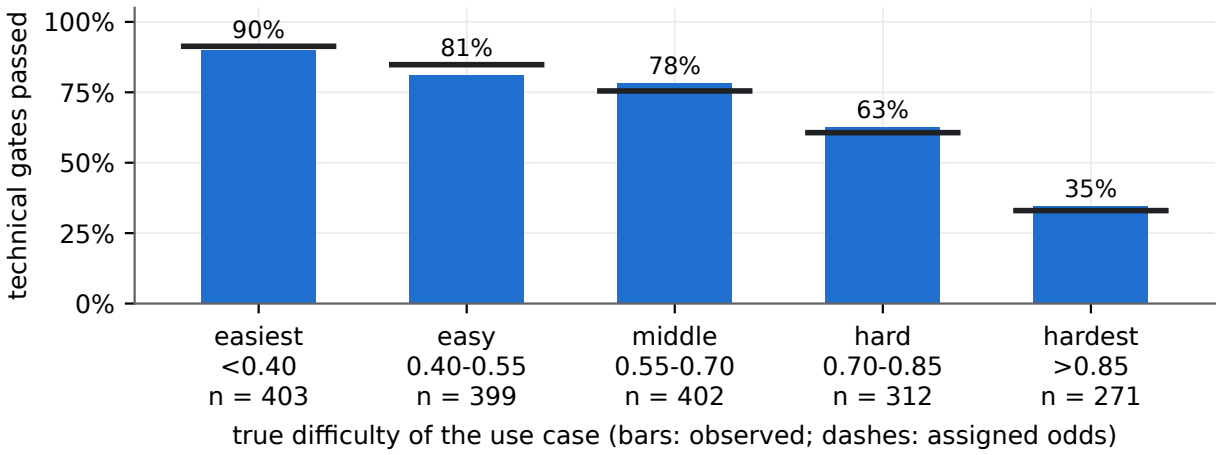


Figure 8: Technical gate outcomes by true difficulty across 1,787 gates in 30 base markets.

Adoption follows S-curves in every sector (Figure 7), consistent with classic diffusion models [2]. We count adoption as *deployments*: one enterprise subscribed to one use case. By month 120, cus-

tomer operations has 145 active deployments (30% of its 481 possible) and software engineering 124 (30% of 413), but the regulated sector only 89 (23% of 384) despite the largest budgets. Adoption plateaus well below its potential because only use cases with a live product can be adopted, and budgets, value hurdles and each product's sales capacity limit the rest. The lag is about reliability, not demand: 46 of 51 failed regulated commercial gates failed on reliability, against none in the other sectors.

Technical success fell from 90% for the easiest use cases to 35% for the hardest, within four points of the assigned odds in every band (Figure 8). In 249 cases where several companies launched the same use case (3.8 on average), the first launcher ended with 32% of customers, about an even split (34%): no lasting first-mover advantage when every firm builds on the same models [10]. Being early in the market did pay: founding companies created $10.4M of value on average with 86% survival, against $4.6M and 77% for the 298 later entrants.

6.8 When the supply chain breaks

In the compute-crunch regime, fab shortages are three times as likely, deeper (50–80% of output) and longer (6–12 months), data centres start with 42% less capacity, and new fab lines take 24 months.

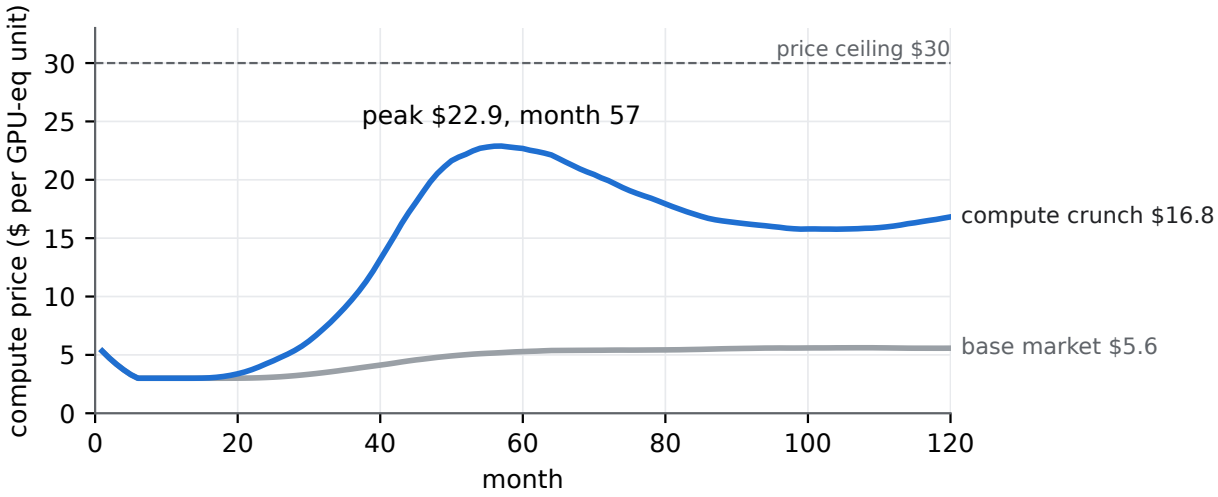


Figure 9: Compute price, mean of 30 markets per regime.

Compute prices peak at 4.4 times the base market and stay three times higher to the end (Figure 9), because new capacity arrives slowly, a delayed and amplified response reminiscent of the bullwhip effect [9, 19]. The served share of compute demand falls to 82% by month 36, cancellations rise 40%, bankruptcies triple (8.9 against 3.0 per market), and total value created for enterprises, AI companies and suppliers falls 38%, from $786M to $484M per market.

The agent struggles here (Figure 10). It never trained on shocks this severe, a case of *distribution shift*: a learned policy is only as good as the situations it has experienced. It kept running three projects on standard and premium budgets, with 42% of starts on the expensive frontier model. The cash-preserving rule earned the most in this mixed population ($4.6M per company). Risk appetite also reversed: among agent-run companies, balanced ones earned most in the base market ($10.8M, against $8.6M cautious and $9.0M aggressive), while under the crunch cautious companies earned

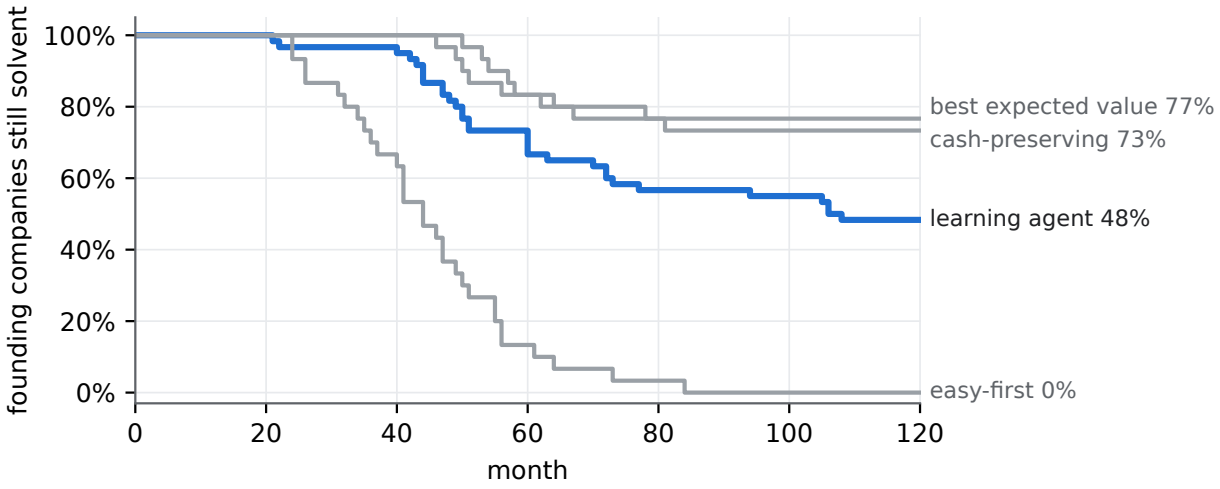


Figure 10: Share of founding companies still solvent in 30 compute-crunch markets with a mixed population.

three times as much as aggressive ones ($1.3M against $0.4M) and survived more often (60% against 48%). Training on randomised conditions, including deliberate shocks, is the standard remedy [23].

7 From simulated findings to real-world hypotheses

The experiments in Section 6 establish consequences of the model's assumptions, not facts about actual AI markets. Reliability thresholds, competition rules, capacity delays, confidence dynamics and every price were supplied by us; the simulation shows what follows if they hold. We therefore state each possible real-world implication as a hypothesis, name the modelled assumptions it rests on, and say what evidence would test it (Table 5). Where an ablation switched a mechanism off (Section 6.6), we know that the finding depends on that mechanism within the model; for the others, the dependence is inferred from the model's structure. Two findings (H1 and H9) concern the learning method rather than the market, and their transfer to people and organisations is an analogy, not a result.

Read this way, the findings are best treated as a map of where to look. The most robust within the model are those an ablation, a sample-size analysis or replication across training runs confirms (H2, H7, H8, H9). The most assumption-bound are those whose magnitude follows directly from parameters we chose (H3, H5). Where a mechanism has a documented real-world counterpart, the direction agrees with prior work, for example amplified supply-chain responses [9, 19] and S-shaped diffusion [2]; the magnitudes have not been calibrated against data and should not be quoted as forecasts.

8 Limitations

Every parameter is an illustrative choice rather than an estimate from data, and the model has not been calibrated or validated against real market data, so magnitudes mean nothing outside the model; directions of effects are more trustworthy, especially where an ablation isolates the mechanism. Training-seed robustness was measured with five runs in two regimes; the shifted-

	Simulated finding	Depends on these modelled assumptions	Evidence needed in real markets
H1	Pure RL learned to do nothing; imitation followed by RL succeeded.	Payoffs arrive 8–12 months after spending, behind a fixed sequence of stages.	Analogy only: studies of how early cash-flow evaluation affects continuation of multi-stage innovation projects.
H2	The original agent accepted twice the technical failures for harder, less contested use cases; across five training runs, agents earned 11% more than the best rule on average.	Success odds fall smoothly with difficulty; value is independent of difficulty; each rival cuts price 12%. Ablation: the advantage reverses without competition.	Project-level data linking difficulty, failure, competition and returns across AI deployments.
H3	The regulated sector adopted slowest despite the largest budgets; 46 of 51 failed pilots failed on reliability.	An 84–92% reliability floor applied as a hard pass/fail gate; model reliability rising about one point per release.	Adoption timing in regulated industries against the measured reliability of available models.
H4	Pilots offered to unready sectors were refused 7.2 times per company (1.6 for the agent).	Confidence is shared and diffuses within a sector and scales adoption. Ablation: removing it changes the agent's lead little.	Whether one vendor's failure measurably lowers other vendors' pilot acceptance in the same sector.
H5	A fab shortage raised compute prices 4.4× and they stayed 3× higher; value created fell 38%.	18–24-month capacity lead times; a bounded price rule driven by utilisation; shock frequency and depth chosen by us.	Compute price and availability around real capacity shocks; downstream failure rates.
H6	The first launcher kept 32% of customers against a 34% even split; founding firms earned 2.3× later entrants.	All firms access the same models; customers compare offers on value; a finite set of use cases.	Market share by launch order for comparable products built on shared foundation models.
H7	All five agents matched or beat the best rule in calm markets but trailed it under a supply crunch; the best risk appetite also reversed.	Regime parameters we specified; agents trained only in the calm regime.	Strategy performance across real periods of abundant and scarce compute.
H8	Without rivals, expected-value ranking beat the learner by $5.2M.	Competition modelled as price discounts and customer choice. Ablation: exclusivity removes the learner's edge.	As H2, plus whether selection rules that ignore rivals underperform in crowded markets.
H9	One agent with one sampling draw looked decisively better (+$5.9M); five training runs with averaged sampling gave +$1.5M.	Variance of this simulator, the training procedure and action sampling.	Methodological; consistent with RL evaluation practice [1, 7]. Transfer to small samples of business pilots is an analogy.

Table 5: Each simulated finding, the assumptions it rests on, and the evidence a real-world claim would require.

regime results, the behaviour analysis and the ablations use a single agent, and five runs give only a rough estimate of between-run spread. The agent was initialised by imitating the best-expected-value rule, so some of its behaviour is inherited. Rankings depend on the rivals present: best expected value beat cash-preserving in the crunch against rule-based rivals but lost to it in mixed populations, and no result here is an equilibrium. The agent was trained on a risk-adjusted version of its score while we report unadjusted value. Prices are a fixed share of value, enterprises do not switch providers mid-contract, upstream firms are scripted and always solvent, and model providers do not act strategically. Finally, the agent never chose to scale a product, which we cannot attribute to a learning gap or to the parameter choice.

9 Conclusion

A transparent simulation of the AI value chain reproduces recognisable dynamics: S-shaped adoption, reliability-gated regulated markets, sticky supply shocks and fragile first-mover advantages. Reinforcement learning, initialised by imitation and refined by PPO on a laptop, reliably matched and usually modestly exceeded strong hand-written rules in calm simulated markets, by avoiding crowded opportunities and timing customer engagement, but reliably underperformed under a supply shock it had not trained on. Accounting for training and sampling randomness shrank an apparently decisive advantage to a modest one, a reminder that single-run RL results can mislead. We hope the simulator serves both as a teaching tool for reinforcement learning and as a sandbox for generating hypotheses about AI markets that real data must then test.

Code and data availability

The simulator, trained agents, evaluation data, figure scripts and a live viewer in which agents play without human input are available at `https://github.com/ravikadam/ai-economy-lab`. Built with PyTorch [12], with Gymnasium and PettingZoo adapters [21, 24].

Acknowledgements

The simulator code, experiments and drafting of this paper were carried out with substantial assistance from Claude (Anthropic), an AI system. The author designed the study, directed the work, and reviewed the results and text, and takes full responsibility for the content.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[2] Frank M. Bass. A new product growth for model consumer durables. *Management Science*, 15(5):215–227, 1969.

[3] Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.

[4] Karl Cobbe, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman. Quantifying generalization in reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

[5] A. J. Faria, David Hutchinson, William J. Wellington, and Steven Gold. Developments in business gaming: A review of the past 40 years. *Simulation & Gaming*, 40(4):464–487, 2009.

[6] J. Doyne Farmer and Duncan Foley. The economy needs agent-based modelling. *Nature*, 460: 685–686, 2009.

[7] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[8] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022.

[9] Hau L. Lee, V. Padmanabhan, and Seungjin Whang. Information distortion in a supply chain: The bullwhip effect. *Management Science*, 43(4):546–558, 1997.

[10] Marvin B. Lieberman and David B. Montgomery. First-mover advantages. *Strategic Management Journal*, 9(S1):41–58, 1988.

[11] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the International Conference on Machine Learning (ICML)*, 1999.

[12] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[13] Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

[14] Everett M. Rogers. *Diffusion of Innovations*. Free Press, 5th edition, 2003.

[15] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[16] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[18] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[19] John D. Sterman. Modeling managerial behavior: Misperceptions of feedback in a dynamic decision making experiment. *Management Science*, 35(3):321–339, 1989.

[20] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[21] J. K. Terry, Benjamin Black, Nathaniel Grammel, et al. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[22] Leigh Tesfatsion. Agent-based computational economics: A constructive approach to economic theory. In Leigh Tesfatsion and Kenneth L. Judd, editors, *Handbook of Computational Economics*, volume 2, pages 831–880. Elsevier, 2006.

[23] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[24] Mark Towers, Ariel Kwiatkowski, Jordan Terry, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[25] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575:350–354, 2019.

[26] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 35, 2022.

[27] Stephan Zheng, Alexander Trott, Sunil Srinivasa, David C. Parkes, and Richard Socher. The AI economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances*, 8(18):eabk2607, 2022.

A Reproducibility

All results run on one laptop. Training and imitation markets use seeds of 1,000,000 and above (each training run shifts them by 10,000,000 times its seed); development markets are 1000–1004 and evaluation sets are 100–129, 200–229 and 300–399, so no evaluation market is ever used in training. Every parameter lives in YAML files under `config/`.

Step	Command (approximate time on an Apple M2 Pro)
Acceptance tests	`pytest` (4 s)
Train (imitation, then PPO)	`python -m training.train -config config/train.yaml` (8 min)
30-market evaluation, regimes, ablations	`python -m training.evaluate -seeds 100:129 -scenarios "config/scenarios/*.yaml" -ablations` (6 min)
100-market evaluation	`python -m training.evaluate -seeds 300:399 -scenarios config/scenarios/compute_crunch.yaml` (5 min)
Market-dynamics experiments	`python -m scripts.experiments -seeds 200:229`; `python scripts/paper_stats.py` (<1 min)
Figures	`python paper/make_figures.py`
Live viewer	`streamlit run viewer/app.py`

Key settings: 120 months; 5 initial companies, at most 12, entry from month 12; 300 enterprises; 60 use cases, 24 visible at start; \$1.5M capital and \$400k credit per company; 3 projects per company; 12 candidate use cases per observation. PPO: learning rate 10^{-4} after cloning, clip 0.2, 4 epochs, minibatch 4,096, entropy coefficient 0.01, GAE $\lambda = 0.95$, $\gamma = 0.99$, 32 worlds $\times$ 60-month rollouts.